

로그인 화면 코드 설명 - 이주영

```
const loginScreen = ({ onLogin }) => {  
  const [username, setUsername] = useState('');  
  const [password, setPassword] = useState('');  
  const [error, setError] = useState('');  
  
  const handleLogin = (e) => {  
    e.preventDefault();  
    setError('');  
  
    // ⚡ ID/PW: netfriend/netfriend로 인증  
    if (username === 'netfriend' && password === 'netfriend') {  
      onLogin(username);  
    } else {  
      setError('유효하지 않은 사용자 이름 또는 비밀번호입니다.');    }  
  };  
};
```

```
return (  
  <div className={styles.loginContainer}>  
    <form onSubmit={handleLogin} className={styles.loginForm}>  
      <h2>AWS 모니티링 데시보드 로그인</h2>  
      {error && <div className={styles.loginError}>{error}</div>}  
      <input  
        type="text"  
        placeholder="사용자 이름 (netfriend)"  
        value={username}  
        onChange={(e) => setUsername(e.target.value)}  
        required  
      />  
      <input  
        type="password"  
        placeholder="비밀번호 (netfriend)"  
        value={password}  
        onChange={(e) => setPassword(e.target.value)}  
        required  
      />  
      <button type="submit" className={styles.loginButton}>로그인</button>  
    </form>  
  </div>  
);
```

로그인 화면은 React의 State를
이용해 ID와 PW를 관리하며, handlelogin함수
에서 하드코딩된 값 ID & 패스워드를
netfriend로 사용자를 검증합니다.
성공 시 부모 컴포넌트에 로그인 이벤트를 전
달합니다.

아래는 로그인 화면 구성으로 당장은 접속이
쉽게 화면에 ID와 패스워드가
보이도록 구현하였다.

웹 대시보드 코드 설명 - 이주영

```
@app.get("/api/metrics/{instance_id}")
async def get_instance_metrics(instance_id: str):
    try:
        end_time = datetime.now(timezone.utc)
        start_time = end_time - timedelta(hours=1)
        dimensions = [{"Name": "InstanceId", "Value": instance_id}]
        period = 300 # 5분 간격

        cpu_metrics = cloudwatch_client.get_metric_statistics(
            Namespace='AWS/EC2', MetricName='CPUUtilization', Dimensions=dimensions,
            StartTime=start_time, EndTime=end_time, Period=period, Statistics=['Average']
        )

        return {
            "success": True,
            "instance_id": instance_id,
            "metrics": format_metrics(cpu_metrics['Datapoints'], '%'),
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"CloudWatch Metric Error: {str(e)}")
```

CPU 모니터링은 CloudWatchAPI를 통해 5분 단위평균 CPU 사용률 데이터를 가져옵니다. 데이터를 그래프 출력력을 위해 시분 형식으로 가공하여 실시간 사용 추이를 시각화합니다.

```
@app.get("/api/logs/{instance_id}")
async def get_instance_log(instance_id: str):
    try:
        response = ec2_client.get_console_output(InstanceId=instance_id, Latest=True)
        if 'Output' in response and response['Output']:
            # Base64로 인코딩된 로그 데이터를 디코딩 (AWS 정책)
            log_data = base64.b64decode(response['Output']).decode('utf-8')
        else:
            log_data = "로그 데이터를 찾을 수 없거나 인스턴스가 실행 중이 아닙니다."
        return {"success": True, "instance_id": instance_id, "log": log_data}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Log Retrieval Error: {str(e)}")
```

시스템 로그 조회는 EC2 API를 사용하여 콘솔 출력을 가져옵니다. AWS 정책에 따라 Base64로 인코딩된 데이터를 받아, 이를 디코딩하여 사용자가 읽을 수 있는 일반 텍스트로 화면에 표시합니다.

웹 대시보드 코드 설명 - 이주영

```
<thead>
  <tr>
    <th>Name</th>
    <th>ID</th>
    <th>상태</th>
    <th>시스템 체크</th>
    <th>IP 주소</th>
    <th>제어</th>
  </tr>
```

Name, ID, 상태, 시스템 체크, IP 주소, 제어필드를 정의합니다

```
<div className={styles.instanceTableContainer}>
  <p className={styles.updateTime}>
```

CSS (App.module.css)에서 파란색 헤더와 기본적인 표 디자인을 정의합니다.

```
{status.instances.map((instance) => {
  const isRunning = instance.InstanceState === 'running';
  const statusClass = isRunning ? styles.running : styles.stopped;
  const rowClass = selectedInstanceId === instance.InstanceId ? styles.selectedRow : '';
```

isRunning값에 따라 **녹색**(styles.running) 또는 **빨간색**(styles.stopped) CSS 클래스를 적용하여 상태를 강조합니다.

```
{instance.SystemStatus === 'ok' ? '✅' : '❌'}
```

'ok'일 경우 **✅**를, 아닐 경우 **❌**를 표시하여 인스턴스의 건강 상태를 직관적으로 나타냅니다.

```
{
  isRunning ? (
    <button className={styles.stopButton} onClick={(e) => { e.stopPropagation(); controlInstance(instance.InstanceId, 'stop'); }}>중지</button>
  ) : (
    <button className={styles.startButton} onClick={(e) => { e.stopPropagation(); controlInstance(instance.InstanceId, 'start'); }} disabled={instance.InstanceState === 'pending' || instance}
  )
}
```

사용자는 인스턴스 상태에 맞는 하나의 버튼만 보게 됩니다. 버튼 클릭시 인스턴스를 시작하거나 중지합니다.

지난 발표까지 진행상황 -이주영

- 실시간 EC2 인스턴스 상태 및 시스템 체크표시.
- 인스턴스별 CPU 사용량 그래프 및 시스템 로그상세 조회.
- 버튼 클릭 한 번으로 인스턴스 시작/중지/원격 제어 기능 구현.

Name	ID	상태	시스템 체크	IP 주소	제어
test-server 01	i-010d5768f67da957	STOPPED	✖	N/A	<button>시작</button>

시스템 로그

로그 조회 오류: HTTP error! Status: 500

CloudWatch 메트릭 (CPU 사용률)

↪ CPU (%)

현재까지 추가한 것 -이주영

- IDS 알림을 대시보드에 심각도별 색상 구분하여 표시.
- 공격자 IP에 대해 수동 차단/해제기능을 제공하는 Fail2Ban 패널 통합.
- FastAPI를 통한 원격 EC2의 실제 방화벽 규칙 변경명령 전송 구현.

실시간 IDS 보안 알림 (3건)

15:28:07	높음
ET SCAN SSH Brute Force attempt SRC: 203.0.113.5 DEST: 192.0.2.10	
15:23:07	낮음
HTTP Protocol Anomaly SRC: 10.0.0.1 DEST: 192.0.2.10	
15:18:07	중간
Suspicious internal traffic flow SRC: 172.16.0.5 DEST: 10.0.1.20	

Fail2Ban 차단 IP 목록

수동 차단할 IP 주소 입력 (예: 1.2.3.4)

수동 차단 (1시간)

IP 주소	이유	남은 시간	해제
1.2.3.4	sshd_mock	23:40:09	해제

-> 현재 IP 차단이 제대로 안 되는 것으로 추정 중

웹 대시보드 코드 설명 - 이주영

```
<div className={styles.idPanel}>
  <h3>실시간 IDS 보안 알림 ({alerts.length}건)</h3>
  <div className={styles.alertsList}>
    {alerts.map((alert, index) => (
      <div key={index} className={` ${styles.alertItem} ${getSeverityClass(alert.alert.severity)} `}>
        <div className={styles.alertHeader}>
          <span className={styles.alertTime}>{formatLocalTime(alert.timestamp)}</span>
          <span className={styles.alertSeverity}>
            {alert.alert.severity <= 1 ? '높음' : alert.alert.severity === 2 ? '중간' : '낮음'}
          </span>
        </div>
        <p className={styles.alertSignature}>{alert.alert.signature}</p>
        <p className={styles.alertDetail}>
          <span className={styles.alertIP}>SRC: {alert.src_ip}</span> |
          <span className={styles.alertIP}>DEST: {alert.dest_ip}</span>
        </p>
      </div>
    ))}
  </div>
</div>
```

- 알림 데이터의 severity값(1, 2, 3)을 기준으로 동적으로 css클래스를 할당하여 시각적 구분을 제공합니다.
- 심각도 1(높음)은 빨간색으로, 심각도 2(중간)는 주황색으로, 심각도 3(낮음)은 녹색으로 표시됩니다
- 알림이 많아질 경우 패널이 화면을 넘치지 않도록 최대 높이(300px)를 설정하고 스크롤 기능을 적용했습니다

Fail2Ban 관련코드 -이주영

```
API_KEY = os.environ.get("API_KEY", "MY_SUPER_API_KEY")
EC2_URL = os.environ.get("FAIL2BAN_EC2_URL", "http://<EC2_공인_IP>:9090")
DB = "data.db"

def init_db():
    conn = sqlite3.connect(DB)
    cur = conn.cursor()
    cur.execute("""CREATE TABLE IF NOT EXISTS blocked(
        ip TEXT PRIMARY KEY,
        reason TEXT,
        expire_at INTEGER
    )""")
    now = int(time.time())
    one_day = 86400
    cur.execute("INSERT OR REPLACE INTO blocked VALUES (?, ?, ?)", ("1.2.3.4", "sshd_mock", now + one_day))
    conn.commit()
    conn.close()

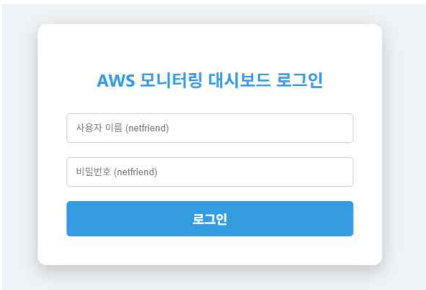
init_db()
class IPCmd(BaseModel):
    ip: str
```

```
def ban_ec2(ip: str):
    headers = {"X-Api-Key": API_KEY}
    try:
        requests.post(f"{EC2_URL}/remote/ban", headers=headers, json={"ip": ip}, timeout=5)
    except Exception as e:
        print(f"EC2 원격 연결 실패 (BAN): {e}")

def unban_ec2(ip: str):
    headers = {"X-Api-Key": API_KEY}
    try:
        requests.post(f"{EC2_URL}/remote/unban", headers=headers, json={"ip": ip}, timeout=5)
    except Exception as e:
        print(f"EC2 원격 연결 실패 (UNBAN): {e}")
```

웹사이트 -이주영

- 로그인/로그아웃 기능을 추가하여 보안강화 및 앞으로 추가될 보안시스템과 연동 될 수 있도록 하였습니다.



```
const LoginScreen = ({ onLogin }) => {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

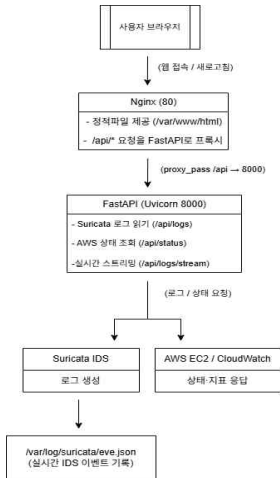
  const handleLogin = (e) => {
    e.preventDefault();
    setError('');

    // ⚠ ID/PW: netfriend/netfriend로 검증
    if (username === 'netfriend' && password === 'netfriend') {
      onLogin(username);
    } else {
      setError('유효하지 않은 사용자 이름 또는 비밀번호입니다.');
```

```
    }
  };

  return (
    <div className={styles.loginContainer}>
      <form onSubmit={handleLogin} className={styles.loginForm}>
        <h2>AWS 모니터링 대시보드 로그인</h2>
        {error && <div className={styles.loginError}>{error}</div>}
        <input
          type="text"
          placeholder="사용자 이름 (netfriend)"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          required
        />
        <input
          type="password"
          placeholder="비밀번호 (netfriend)"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />
        <button type="submit" className={styles.loginButton}>로그인</button>
      </form>
    </div>
  );
};
```

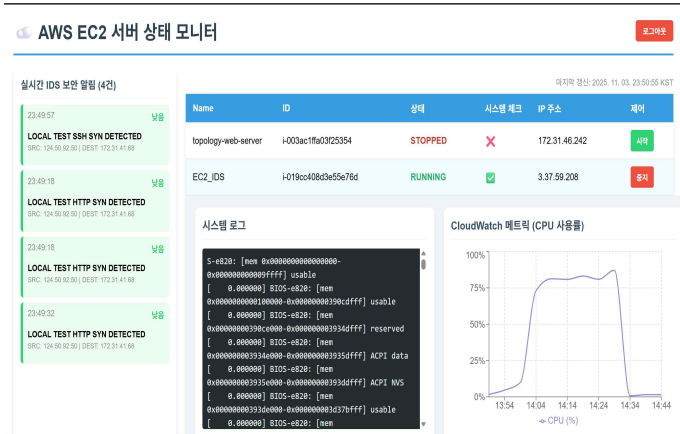

전체 흐름도 - 박혜소



- 사용자 브라우저 → 웹 접속 및 /api 요청 전송
- Nginx (80) → 정적 파일 제공, /api 요청을 FastAPI로 프록시
- FastAPI (8000) → Suricata 로그(eve.json) 가공 및 AWS 상태 조회
- Suricata IDS → 네트워크 트래픽 탐지 후 로그 생성
- AWS EC2 / CloudWatch → 서버 상태·지표 데이터 제공
- 결과 → IDS 탐지 로그와 서버 상태가 실시간 대시보드로 시각화됨

브라우저 (사용자)	웹 대시보드 접근 및 로그 확인
Nginx (80)	정적파일 제공 및 API 프록시
FastAPI (8000)	Suricata 로그 가공 및 AWS 상태 조회
Suricata (IDS)	네트워크 트래픽 탐지 및 알람 생성
AWS (EC2, CloudWatch)	서버 상태 및 모니터링 지표 제공

웹 대시보드 시각화 - 박혜소



패키지 설치 - 박혜소

1. 패키지 목록 갱신

```
last login: Wed Oct 13 12:41:10 UTC 2020 from 210.119.103.137
ubuntu@ip-172-31-41-68:~$ sudo apt update
Hit:1 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1498 kB]
Get:5 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [288 kB]
Get:6 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:8 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [15.3 kB]
Get:9 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1489 kB]
Get:10 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [288 kB]
```

sudo apt update

2. 패키지 설치

```
ubuntu@ip-172-31-41-68:~$ sudo apt install -y git python3-venv python3-pip nginx jq acl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
python3-pip is already the newest version (24.0+dfsg-1ubuntu1.3).
nginx is already the newest version (1.24.0-2ubuntu7.5).
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
The following additional packages will be installed:
  python3-pip-whl python3-setuptools-whl python3.12-venv
The following NEW packages will be installed:
  acl python3-pip-whl python3-setuptools-whl python3-venv python3.12-venv
0 upgraded, 5 newly installed, 0 to remove and 13 not upgraded.
Need to get 2469 kB of archives.
```

sudo apt install -y git python3-venv python3-pip nginx jq acl

- git: 코드 가져오기
- python3-venv/pip: 백엔드 의존성 격리/설치
- nginx: 웹/프록시 서버
- jq: JSON 보기 좋게
- acl: eve.json 읽기 권한 세밀하게 부여

- 관련 패키지-

코드 다운로드 및 가상환경 구성 - 박혜소

1. 깃허브에서 코드 내려받기

```
ubuntu@ip-172-31-41-68:~$ cd -
ubuntu@ip-172-31-41-68:~$ git clone https://github.com/sh456783/NetFriend.git server-monitor
Cloning into 'server-monitor':
remote: Enumerating objects: 44, done.
remote: Counting objects: 100% (44/44), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 44 (delta 1), reused 29 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (44/44), 4.90 MiB | 23.89 MiB/s, done.
Resolving deltas: 100% (1/1), done.
ubuntu@ip-172-31-41-68:~$
```

git clone <https://github.com/sh456783/NetFriend.git> server-monitor

2. 가상환경 생성 및 활성화

```
ubuntu@ip-172-31-41-68:~$ python3 -m venv venv
ubuntu@ip-172-31-41-68:~$ source venv/bin/activate
(venv) ubuntu@ip-172-31-41-68:~$
```



```
(venv) ubuntu@ip-172-31-41-68:~$
```

(venv) - 활성화

```
python3 -m venv venv
source venv/bin/activate
```

3. FastAPI 및 의존성 설치

```
(venv) ubuntu@ip-172-31-41-68:~$ pip install fastapi uvicorn[standard] boto3
Collecting fastapi
  Downloading fastapi-0.119.0-py3-none-any.whl.metadata (28 kB)
Collecting boto3
  Downloading boto3-1.40.52-py3-none-any.whl.metadata (6.7 kB)
Collecting uvicorn[standard]
  Downloading uvicorn-0.37.0-py3-none-any.whl.metadata (6.6 kB)
Collecting starlette<0.49.0,>=0.40.0 (from fastapi)
  Downloading starlette-0.48.0-py3-none-any.whl.metadata (6.3 kB)
Collecting pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4 (from fastapi)
  Downloading pydantic-2.12.2-py3-none-any.whl.metadata (85 kB)
85.8/85.8 kB 6.4 MB/s eta 0:00:00
```

pip install fastapi "uvicorn[standard]"

4. suricata 로그 권한

```
ubuntu@ip-172-31-41-68:~$ sudo usermod -aG suricata ubuntu
ubuntu@ip-172-31-41-68:~$ sudo setfacl -m u:ubuntu:r /var/log/suricata/eve.json
ubuntu@ip-172-31-41-68:~$ sudo setfacl -m u:ubuntu:rx /var/log/suricata
ubuntu@ip-172-31-41-68:~$
```

```
sudo usermod -aG suricata ubuntu
sudo setfacl -m u:ubuntu:r /var/log/suricata/eve.json
sudo setfacl -m u:ubuntu:rx /var/log/suricata
```

FastAPI 서버 실행 - 박혜소

1. 서버 수동 실행

```
ubuntu@ip-172-31-41-68:~$ python3 main.py
INFO: Started server process [1882]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

uvicorn main:app --host 0.0.0.0 --port 8000

2. systemd 자동 실행 등록

```
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$ sudo tee /etc/systemd/system/server-monitor.service >/dev/null << EOF
> [Unit]
> Description=Server Monitor Backend (FastAPI)
> After=network.target
>
> [Service]
> User=ubuntu
> # main.py가 있는 디렉터리
> WorkingDirectory=/home/ubuntu/server-monitor
> # 이 venv의 uvicorn 경로 (= which uvicorn 결과와 동일)
> ExecStart=/home/ubuntu/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
> Restart=always
> Environment=AWS_DEFAULT_REGION=ap-northeast-2
>
> [Install]
> WantedBy=multi-user.target
> EOF
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$
```

sudo tee /etc/systemd/system/server-monitor.service >/dev/null <<EOF

```
[Unit]
Description=Server Monitor Backend (FastAPI) # 서비스 이름 및 설명
After=network.target # 네트워크 활성화 후 시작하도록 의존성 설정

[Service]
User=ubuntu # 서비스를 실행할 사용자 (보안 계정)
WorkingDirectory=/home/ubuntu/server-monitor # 실행 시 기준이 되는 경로
ExecStart=/home/ubuntu/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
# 가상환경 uvicorn으로 FastAPI 앱을 8000 포트로 실행

Restart=always # 비정상 종료 시 무조건 자동 재시작
Environment=AWS_DEFAULT_REGION=ap-northeast-2 # 서비스 내부에서 사용할 환경 변수 설정

[Install]
WantedBy=multi-user.target # 시스템 부팅 시 자동 시작 서비스로 등록
```

FastAPI 서버 실행 - 박혜소

```
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$ sudo systemctl daemon-reload
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$ sudo systemctl enable --now server-monitor
Created symlink /etc/systemd/system/multi-user.target.wants/server-monitor.service → /etc/systemd/system/server-monitor.service.
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$ sudo systemctl status server-monitor --no-pager
● server-monitor.service - Server Monitor Backend (FastAPI)
   Loaded: loaded (/etc/systemd/system/server-monitor.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-15 20:06:11 UTC; 6s ago
     Main PID: 2133 (uvicorn)
        Tasks: 6 (limit: 1017)
      Memory: 61.4M (peak: 65.5M)
         CPU: 768ms
    CGroup: /system.slice/server-monitor.service
            └─2133 /home/ubuntu/venv/bin/python3 /home/ubuntu/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000

Oct 15 20:06:11 ip-172-31-41-68 systemd[1]: Started server-monitor.service - Server Monitor Backend (FastAPI).
Oct 15 20:06:12 ip-172-31-41-68 uvicorn[2133]: INFO: Started server process [2133]
Oct 15 20:06:12 ip-172-31-41-68 uvicorn[2133]: INFO: Waiting for application startup.
Oct 15 20:06:12 ip-172-31-41-68 uvicorn[2133]: INFO: Application startup complete.
Oct 15 20:06:12 ip-172-31-41-68 uvicorn[2133]: INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$
```

`sudo systemctl status server-monitor --no-pager`

AWS IAM Role 권한 설정 추가 - 박혜소

정책 편집기

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "ec2:DescribeInstances",
8         "ec2:DescribeInstanceStatus",
9         "ec2:DescribeTags",
10        "ec2:GetConsoleOutput"
11      ],
12      "Resource": "*"
13    },
14    {
15      "Effect": "Allow",
16      "Action": [
17        "cloudwatch:GetMetricStatistics",
18        "cloudwatch:ListMetrics"
19      ],
20      "Resource": "*"
21    },
22    {
23      "Effect": "Allow",
24      "Action": [
25        "ec2:StartInstances",
26        "ec2:StopInstances"
27      ],
28      "Resource": "*"
29    }
30  ]
31 }
```

+ 새 문 추가

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      // EC2 인스턴스 조회 권한
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances", // 인스턴스 정보 조회
        "ec2:DescribeInstanceStatus", // 상태 확인
        "ec2:DescribeTags", // 태그(Name 등) 확인
        "ec2:GetConsoleOutput" // 콘솔 출력(로그)
      ],
      "Resource": "*"
    },
    {
      // CloudWatch 메트릭 조회 권한
      "Effect": "Allow",
      "Action": [
        "cloudwatch:GetMetricStatistics", // 통계값 조회
        "cloudwatch:ListMetrics" // 메트릭 목록 확인
      ],
      "Resource": "*"
    },
    {
      // EC2 제어 권한 (시작/중지)
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances", // 인스턴스 시작
        "ec2:StopInstances" // 인스턴스 중지
      ],
      "Resource": "*"
    }
  ]
}
```

정책 세부 정보

정책 이름

이 정책을 식별하는 의미 있는 이름을 입력합니다.

ServerMonitor-Minimum

최대 128자입니다. 영숫자 및 '+', '@', '_' 문자를 사용하세요.

이 정책에 정의된 권한 정보

이 정책 문서에 정의된 권한은 허용되거나 거부되는 작업을 지정합니다. IAM 자격 증명(사용자, 사용자 그룹)

Q 검색

허용(서비스 448개 중 2개)

서비스	액세스 수준	리소스
CloudWatch	제한적: 나열, 읽기	모든 리소스
EC2	제한적: 나열, 읽기, 쓰기	모든 리소스

취소

이전

정책 생성

Suricata 로그 연동 - 박혜소

Suricata 로그 처리 및 실시간 API 구현 추가

1) 로그 경로

```
SURICATA_EVE = os.environ.get("SURICATA_EVE", "/var/log/suricata/eve.json")
```

2) 로그 파일 읽기

```
def _tail_lines(path: str, n: int) -> List[str]:
    """eve.json의 마지막 n줄만 읽기 (대용량 파일도 안전하게)."""
    if not os.path.exists(path):
        return []
    try:
        with open(path, "r", encoding="utf-8", errors="ignore") as f:
            lines = f.readlines()
            return lines[-n:]
    except Exception:
        return []
```

3) 데이터 가공 및 변환

```
def _map_event(ev: Dict[str, Any]) -> Dict[str, Any]:
    alert = ev.get("alert") or {}

    # 원본 UTC timestamp → 한국시간(KST, UTC+9) 변환
    ts_utc = ev.get("timestamp")
    ts_kst = None
    if ts_utc:
        try:
            # Suricata는 ISO 포맷이므로 Z 또는 +00:00 처리
            dt = datetime.fromisoformat(ts_utc.replace("Z", "+00:00"))
            ts_kst = (dt + timedelta(hours=9)).strftime("%Y-%m-%d %H:%M:%S")
        except Exception:
            ts_kst = ts_utc # 파싱 실패 시 원본 그대로 표시

    return {
        "timestamp": ts_kst,
        "signature": alert.get("signature"),
        "severity": alert.get("severity"),
        "src_ip": ev.get("src_ip"),
        "src_port": ev.get("src_port"),
        "dest_ip": ev.get("dest_ip"),
        "dest_port": ev.get("dest_port"),
        "proto": ev.get("proto"),
        "category": alert.get("category"),
    }
```


Suricata 로그 연동 - 박혜소

Suricata 로그 처리 및 실시간 API 구현 추가

3) Suricata 로그 조회 API

```
@app.get("/api/logs")
def get_suricata_logs(n: int = 50, path: Optional[str] = None):
    """
    최근 N건 Suricata alert 반환.
    사용법: GET /api/logs?n=50 (기본 파일: /var/log/suricata/eve.json)
    """
    eve_path = path or SURICATA_EVE
    try:
        out: List[Dict[str, Any]] = []
        for ln in _tail_lines(eve_path, n):
            try:
                ev = json.loads(ln)
                if ev.get("event_type") == "alert":
                    out.append(_map_event(ev))
            except Exception:
                continue
        return out
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"eve_read_error: {e}")
```

4) 실시간 스트리밍

```
@app.get("/api/logs/stream")
def stream_suricata_logs():
    """
    SSE(Simple Server-Sent Events) 방식으로 Suricata alert 실시간 스트리밍.
    사용법: 브라우저/프론트에서 EventSource('/api/logs/stream') 로 수신
    """
    def gen():
        last_size = 0
        while True:
            try:
                size = os.path.getsize(SURICATA_EVE) if os.path.exists(SURICATA_EVE) else 0
                if size > last_size:
                    with open(SURICATA_EVE, "r", encoding="utf-8", errors="ignore") as f:
                        f.seek(last_size)
                        for ln in f:
                            try:
                                ev = json.loads(ln)
                                if ev.get("event_type") == "alert":
                                    yield f"data: {json.dumps(_map_event(ev))}\n\n"
                            except Exception:
                                pass
                        last_size = size
            except Exception:
                pass
            import time
            time.sleep(1.0)
        return Response(gen(), media_type="text/event-stream")
```

FastAPI 연동 확인 - 박혜소

FastAPI <-> AWS 연동 성공!

```
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor$ curl -s "http://127.0.0.1:8000/api/status?role_key=Role&role_value=IDS" | jq .
{
  "success": true,
  "instances": [
    {
      "InstanceId": "i-483ac1ff6d3f2384",
      "Name": "tag1gy-wd-server",
      "PublicIp": null,
      "PrivateIp": null,
      "InstanceType": "t3.micro",
      "InstanceState": "stopped",
      "SystemStatus": "not-applicable",
      "InstanceStatus": "not-applicable",
      "LastUpdated": "2025-10-16 16:58:36 UTC"
    },
    {
      "InstanceId": "i-419cc48d1e5e76d",
      "Name": "EC2_IDS",
      "PublicIp": null,
      "PrivateIp": null,
      "InstanceType": "t3.micro",
      "InstanceState": "running",
      "SystemStatus": "ok",
      "InstanceStatus": "ok",
      "LastUpdated": "2025-10-16 16:58:36 UTC"
    }
  ]
}
```

curl -s "http://127.0.0.1:8000/api/status?role_key=Role&role_value=IDS" | jq .

FastAPI <-> Suricata 연동성공!

```
3138
ubuntu@ip-172-31-41-68:~/server-monitor$ echo '{"timestamp":"2025-10-23T08:42:31.000000+00:00","event_type":"alert","src_ip":"203.0.113.5","dest_ip":"172.31.41.68","proto":"TCP","alert":{"signature":"LOCAL TEST SSH DETECTED"},"category":"Test Alert","severity":2}}' \
| sudo tee -a /var/log/suricata/eve.json >/dev/null
ubuntu@ip-172-31-41-68:~/server-monitor$ curl -s "http://127.0.0.1:8000/api/logs?n=5" | jq .
[
  {
    "timestamp": "2025-10-23 17:42:31",
    "signature": "LOCAL TEST SSH DETECTED",
    "severity": 2,
    "src_ip": "203.0.113.5",
    "src_port": null,
    "dest_ip": "172.31.41.68",
    "dest_port": null,
    "proto": "TCP",
    "category": "Test Alert"
  }
]
```

curl -s "http://127.0.0.1:8000/api/logs?n=5" | jq .

Suricata 로그 연동 - 박혜소

Nginx 프록시 설정

```
location /api/ {
    proxy_pass http://127.0.0.1:8000/;    # 백엔드 uvicorn
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

#SSE 스트림 사용 시
location /api/logs/stream {
    proxy_pass http://127.0.0.1:8000/api/logs/stream;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header Host $host;
    proxy_buffering off;
    proxy_read_timeout 3600s;
    chunked_transfer_encoding on;
}
```

sudo nano /etc/nginx/sites-available/default

FastAPI 연동 확인 - 박혜소

FastAPI <-> Nginx 연동 성공!

```
ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ curl -sS http://127.0.0.1:8000/api/status | head
{"success":true,"instances":[{"InstanceId":"i-003ac1ffa03f25354","Name":"topology-web-server","PublicIp":null,"PrivateIp":"172.31.46.242","InstanceType":"t2.micro","InstanceState":"stopped","SystemStatus":"not-applicable","InstanceStatus":"not-applicable","LastUpdated":"2025-10-17 03:47:03 UTC"},{"InstanceId":"i-019cc408d3e55e76d","Name":"EC2_IDS","PublicIp":"3.37.59.208","PrivateIp":"172.31.41.68","InstanceType":"t3.micro","InstanceState":"running","SystemStatus":"ok","InstanceStatus":"ok","LastUpdated":"2025-10-17 03:47:36 UTC"}]}
ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ curl -sS http://localhost/api/status | head
{"success":true,"instances":[{"InstanceId":"i-003ac1ffa03f25354","Name":"topology-web-server","PublicIp":null,"PrivateIp":"172.31.46.242","InstanceType":"t2.micro","InstanceState":"stopped","SystemStatus":"not-applicable","InstanceStatus":"not-applicable","LastUpdated":"2025-10-17 03:47:36 UTC"},{"InstanceId":"i-019cc408d3e55e76d","Name":"EC2_IDS","PublicIp":"3.37.59.208","PrivateIp":"172.31.41.68","InstanceType":"t3.micro","InstanceState":"running","SystemStatus":"ok","InstanceStatus":"ok","LastUpdated":"2025-10-17 03:47:36 UTC"}]}
ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$
```

curl -sS http://127.0.0.1:8000/api/status | head (백엔드에서 호출)

curl -sS http://localhost/api/status | head (브라우저에서 호출)

로그 시각화 - 박혜소

IDS 로그 시각화 수정

```
// 4. NEW: IDS 알림을 가져오는 함수
const fetchAlerts = async () => {
  try {
    const response = await fetch('http://localhost:8000/api/ids_alert');
    if (!response.ok) throw new Error('HTTP error! Status: ${response.status}');
    const data = await response.json();

    if (data.success && data.alerts) {
      setAlerts(data.alerts);
    }
  } catch (e) {
    console.error("Alerts Fetch failed:", e);
  }
};
```



```
// 4. NEW: IDS 알림을 가져오는 함수 (실제 Suricata 연동)
const fetchAlerts = async () => {
  // 로그인 안 되어 있으면 스킵
  if (!isLoggedIn) return;

  try {
    // Nginx 프록시 통해 백엔드 호출 (상대경로)
    const res = await fetch('/api/logs?n=10', { cache: 'no-store' });
    if (!res.ok) throw new Error('HTTP ${res.status}');
    const raw = await res.json(); // [{timestamp, signature, severity, ...}]

    // 패킷이 기대하는 모양(alert.*, timestamp/src_ip/dest_ip)을 맞춰 [
    const mapped = Array.isArray(raw)
      ? raw.map(ev => ({
        timestamp: ev.timestamp, // 이미 KST 문자열
        src_ip: ev.src_ip || null,
        dest_ip: ev.dest_ip || null,
        alert: {
          signature: ev.signature || '-',
          severity: Number(ev.severity ?? 3), // 1=높음, 2=중간, 3=낮
        },
      }))
      : [];

    setAlerts(mapped);
  } catch (e) {
    console.error('IDS 로그 로드 실패:', e);
  }
};
```

프론트엔드 의존성 설치 - 박혜소

Node.js + npm 설치

1. 패키지 목록 갱신

```
(env) ubuntu@1p-172-31-41-68: ~/server-monitor/server-monitor/frontend$ sudo apt update
Hit:1 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu noble InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
13 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
sudo apt update
```

2. node.js와 npm 설치

```
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ sudo apt install -y nodejs npm
```

```
sudo apt install -y node.js npm
```

3. 설치 및 버전 확인

```

[dev] ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/transfer $ node -v
v10.20.8
10.8.2

```

```
node -v && npm -v
```

프론트엔드 빌드 - 박혜소

1. 프론트엔드 빌드 성공

```
added 1381 packages in 2m
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ npm run build

> frontend@0.1.0 build
> react-scripts build

Creating an optimized production build...
Compiled successfully.

File sizes after gzip:

  152.37 kB  build/static/js/main.9fa979ce.js
  1.76 kB    build/static/js/453.8701dc61.chunk.js
  1.22 kB    build/static/css/main.ce42f58d.css

The project was built assuming it is hosted at /.
You can control this with the homepage field in your package.json.

The build folder is ready to be deployed.
You may serve it with a static server:

  npm install -g serve
  serve -s build

Find out more about deployment here:

  https://cra.link/deployment

(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$
```

npm run build

2. Nginx 최종 배포 및 권한 설정

```
ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ sudo rm -rf /var/www/html/*
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ sudo cp -r build/* /var/www/html/
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ sudo chown -R www-data:www-data /var/www/html
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ sudo chmod -R 755 /var/www/html
(venv) ubuntu@ip-172-31-41-68:~/server-monitor/server-monitor/frontend$ sudo systemctl restart nginx
```

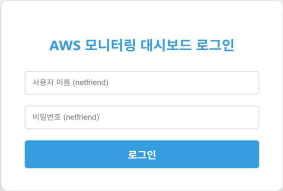
```
sudo rm -rf /var/www/html/*
sudo cp -r build/* /var/www/html/
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html
sudo systemctl restart nginx
```

3. http:// <퍼블릭 IP> 로 접속!!

- 접속 주소: http://3.37.59.208/

웹 대시보드 접속 - 박혜소

접속 주소: <http://3.37.59.208/>



AWS 모니터링 대시보드 로그인

사용자 이름 (netfriend)

비밀번호 (netfriend)

로그인

The image shows a login form for the AWS Monitoring Dashboard. It is centered on a light blue background. The form is a white rounded rectangle with a subtle shadow. It contains the title 'AWS 모니터링 대시보드 로그인' in blue, followed by two input fields for '사용자 이름 (netfriend)' and '비밀번호 (netfriend)', and a blue '로그인' button at the bottom.

웹 대시보드 접속 - 박혜소

AWS EC2 서버 상태 모니터

[로그아웃](#)

실시간 IDS 보안 알림 (200건)

01:36:34 보통

LOCAL TEST HTTP SYN DETECTED
SRC: 124.50.92.50 | DEST: 172.31.41.68

01:36:34 보통

LOCAL TEST HTTP SYN DETECTED
SRC: 124.50.92.50 | DEST: 172.31.41.68

01:35:04 보통

LOCAL TEST HTTP SYN DETECTED
SRC: 172.31.41.68 | DEST: 169.254.169.254

01:35:04 보통

LOCAL TEST HTTP SYN DETECTED
SRC: 172.31.41.68 | DEST: 169.254.169.254

01:35:04 보통

LOCAL TEST HTTP SYN DETECTED

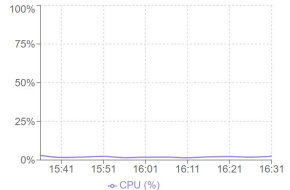
마지막 갱신: 2025. 11. 03. 01:37:55 KST

Name	ID	상태	시스템 체크	IP 주소	제어
topology-web-server	i-003ac1ffa03f25354	STOPPED	✗	172.31.46.242	시작
EC2_IDS	i-019cc408d3e55e76d	RUNNING	✓	3.37.59.208	중지

시스템 로그

```
[ 0.218802] NET: Registered
PF_NETLINK/PF_ROUTE protocol family
[ 0.219204] DMA: preallocated 128 KiB
GFP_KERNEL pool for atomic allocations
[ 0.220121] DMA: preallocated 128 KiB
GFP_KERNEL[GFP_DMA pool for atomic allocations
[ 0.221069] DMA: preallocated 128 KiB
GFP_KERNEL[GFP_DMA32 pool for atomic allocations
[ 0.222066] audit: initializing netlink subsys
(disabled)
[ 0.222872] audit: type=2000
audit(1762097548.011:1): state=initialized
audit_enabled=0 res=1
[ 0.222872] thermal sys: Registered thermal
```

CloudWatch 메트릭 (CPU 사용률)



IDS 연동 확인- 박혜소

```

laavvppl@laavvppl-VirtualBox:~$ sudo hpinglaavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$ sudo hping3 -S -p 22 -c 5 3.37.59.208
[sudo] laavvppl 암호:
최종합니다만, 다시 시도하십시오.
[sudo] laavvppl 암호:
HPING 3.37.59.208 (enp0s3 3.37.59.208): 5 set, 48 headers + 0 data bytes
len=46 ip=3.37.59.208 ttl=64 id=44 sport=22 flags=SA seq=0 win=65535 rtt=24.0 ms
len=46 ip=3.37.59.208 ttl=64 id=45 sport=22 flags=SA seq=1 win=65535 rtt=18.8 ms
len=46 ip=3.37.59.208 ttl=64 id=46 sport=22 flags=SA seq=2 win=65535 rtt=19.0 ms
len=46 ip=3.37.59.208 ttl=64 id=47 sport=22 flags=SA seq=3 win=65535 rtt=25.1 ms
len=46 ip=3.37.59.208 ttl=64 id=48 sport=22 flags=SA seq=4 win=65535 rtt=26.0 ms

--- 3.37.59.208 hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 10.8/21.0/26.0 ms
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$
laavvppl@laavvppl-VirtualBox:~$

```

←

→

↻

주요 알림 3.37.59.208

🔍 ☆

🏠

🔴

⋮

📱

Gmail

지도

Google

다운로드

📁 모든 북마크

🌐

AWS EC2 서버 상태 모니터

로그아웃

현재 탐지된 보안 알림이 없습니다.

마지막 경신: 2025-10-23 20:14:43 UTC

Name	ID	상태	시스템 체크	IP 주소	제어
topology-web-server	i-003ac1ffa03f25354	STOPPED	✖	N/A	시작
EC2_IDS	i-019cc408d3e55e76d	RUNNING	✔	N/A	중지